

前缀和与差分

前言

在处理数组区间问题时，我们经常遇到需要快速查询或更新大量数据的场景。前缀和与差分作为两种经典的数组预处理技术，能够将看似复杂的问题化繁为简。今天，让我们深入探讨这两种技术的原理、应用和优化方法。

第一部分：前缀和（Prefix Sum）

1.1 核心概念

前缀和 是一种经典的数组预处理技术。对于数组 `arr[]`，我们定义前缀和数组 `prefix[]`，其中：

```
prefix[i] = arr[0] + arr[1] + ... + arr[i]
```

换句话说，`prefix[i]` 存储的是从数组开始到第 `i` 个位置的所有元素的累积和。

1.2 一维前缀和

构建方法

```
def build_prefix_sum(arr):
    n = len(arr)
    prefix = [0] * (n + 1) # prefix[0] = 0
    for i in range(n):
        prefix[i + 1] = prefix[i] + arr[i]
    return prefix
```

为什么要开 `n+1` 的空间？ 通过让 `prefix[0] = 0`，我们可以统一处理所有区间查询，避免边界情况的特殊处理。

区间查询

一旦构建好前缀和数组，查询任意区间 `[l, r]` 的和只需 **$O(1)$** 时间：

```
def range_sum(prefix, l, r):
    # 查询原数组 arr[l...r] 的和
    return prefix[r + 1] - prefix[l]
```

时间复杂度分析：

- 预处理： $O(n)$

- 单次查询: $O(1)$
- 总体: $O(n + q)$, 其中 q 为查询次数

1.3 二维前缀和

对于二维矩阵, 前缀和的思想可以扩展。定义 $\text{prefix}[i][j]$ 为以 $(0,0)$ 到 (i,j) 构成的矩形区域的元素之和。

构建方法

```
def build_2d_prefix_sum(matrix):
    m, n = len(matrix), len(matrix[0])
    prefix = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            prefix[i][j] = (matrix[i-1][j-1] +
                           prefix[i-1][j] +
                           prefix[i][j-1] -
                           prefix[i-1][j-1])

    return prefix
```

为什么要减去 $\text{prefix}[i-1][j-1]$? 因为在计算 $\text{prefix}[i-1][j] + \text{prefix}[i][j-1]$ 时, 左上角区域 $\text{prefix}[i-1][j-1]$ 被计算了两次, 需要减回一次。

区间查询

```
def range_sum_2d(prefix, r1, c1, r2, c2):
    # 查询矩形 (r1,c1) 到 (r2,c2) 的和
    return (prefix[r2+1][c2+1] -
            prefix[r1][c2+1] -
            prefix[r2+1][c1] +
            prefix[r1][c1])
```

这里同样使用容斥原理来处理重复计算的区域。

1.4 高维扩展

前缀和的思想可以扩展到任意维度。对于 d 维数组, 构建时的递推关系为:

```
prefix[i1][i2][...] [ia] = arr[i1][i2][...] [ia]
    +  $\Sigma$ (去掉某一维的所有前缀和)
    -  $\Sigma$ (去掉某两维的所有前缀和)
    + ... (交替加减)
```

这遵循容斥原理的模式。

第二部分：差分（Difference Array）

2.1 核心概念

差分 与前缀和互为逆操作。对于数组 `arr[]`，定义差分数组 `diff[]`，其中：

```
diff[i] = arr[i] - arr[i-1]    (i > 0)
diff[0] = arr[0]
```

关键性质：通过对差分数组求前缀和，可以恢复原数组！

2.2 一维差分

构建方法

```
def build_difference(arr):
    n = len(arr)
    diff = [0] * (n + 1)
    diff[0] = arr[0]
    for i in range(1, n):
        diff[i] = arr[i] - arr[i-1]
    diff[n] = -arr[n-1] # 哨兵值
    return diff
```

区间更新

差分的真正威力在于 **高效的区间更新**。如果想让原数组中 `[l, r]` 区间的所有元素都增加 `val`，只需：

```
def range_add(diff, l, r, val):
    diff[l] += val
    diff[r + 1] -= val
```

这样只需 **$O(1)$** 的时间，然后对整个差分数组求前缀和，就能得到更新后的数组。

时间复杂度分析：

- 单次区间更新： $O(1)$
- 所有更新后求解： $O(n)$
- 总体： $O(n + m)$ ，其中 m 为更新次数

2.3 二维差分

对于二维矩阵的区间更新，差分同样适用。

构建方法

```
def build_2d_difference(matrix):
    m, n = len(matrix), len(matrix[0])
    diff = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(m):
        for j in range(n):
            diff[i][j] = matrix[i][j]
            if i > 0:
                diff[i][j] -= matrix[i-1][j]
            if j > 0:
                diff[i][j] -= matrix[i][j-1]
            if i > 0 and j > 0:
                diff[i][j] += matrix[i-1][j-1]

    return diff
```

矩形区域更新

```
def range_add_2d(diff, r1, c1, r2, c2, val):
    diff[r1][c1] += val
    diff[r1][c2 + 1] -= val
    diff[r2 + 1][c1] -= val
    diff[r2 + 1][c2 + 1] += val
```

这利用了容斥原理：四个角的位置分别被增加和减少，确保只有矩形内部被正确更新。

第三部分：应用场景对比

场景选择指南

场景	推荐方案	原因
大量范围查询，少量更新	前缀和	单次查询 $O(1)$ ，更新 $O(n)$
大量范围更新，少量查询	差分	单次更新 $O(1)$ ，查询需重建 $O(n)$
交替查询和更新	线段树/树状数组	两者都是 $O(\log n)$
仅需单点操作	原数组	无需预处理

实战例题

例题 1：区间和查询（LeetCode 303）

问题：给定数组，支持多次范围和查询。

解法：使用前缀和

```
class NumArray:
    def __init__(self, nums):
        self.prefix = [0]
        for num in nums:
            self.prefix.append(self.prefix[-1] + num)

    def sumRange(self, left, right):
        return self.prefix[right + 1] - self.prefix[left]
```

例题 2：航班预订系统 (LeetCode 1109)

问题：处理多个区间增加操作，最后查询每个座位的总预订数。

解法：使用差分

```
def corpFlightBookings(bookings, n):
    diff = [0] * (n + 1)

    # 差分更新
    for first, last, seats in bookings:
        diff[first - 1] += seats
        diff[last] -= seats

    # 前缀和恢复
    result = []
    current = 0
    for i in range(n):
        current += diff[i]
        result.append(current)

    return result
```

第四部分：进阶技巧

4.1 前缀和的其他应用

子数组和问题

求满足特定条件的子数组个数：

```
def subarraySum(nums, k):
    # 使用前缀和 + 哈希表
    prefix_sum = 0
    count_map = {0: 1}
```

```
result = 0

for num in nums:
    prefix_sum += num
    # 如果存在前缀和为 prefix_sum - k, 则找到一个满足条件的子数组
    if prefix_sum - k in count_map:
        result += count_map[prefix_sum - k]

    count_map[prefix_sum] = count_map.get(prefix_sum, 0) + 1

return result
```

时间复杂度: $O(n)$

最大子数组和 (Kadane 算法)

```
def maxSubArray(nums):
    max_sum = current_sum = nums[0]

    for i in range(1, len(nums)):
        current_sum = max(nums[i], current_sum + nums[i])
        max_sum = max(max_sum, current_sum)

    return max_sum
```

4.2 差分的其他应用

区间覆盖问题

```
def removeDuplicates(intervals):
    # 使用差分处理区间覆盖
    events = []
    for start, end in intervals:
        events.append((start, 1))    # 区间开始
        events.append((end, -1))     # 区间结束

    events.sort()

    result = []
    count = 0
    last_start = 0

    for pos, delta in events:
        if count > 0 and pos > last_start:
            result.append((last_start, pos))

        count += delta
        last_start = pos
```

```
return result
```

4.3 模运算下的前缀和

在某些竞赛问题中，前缀和可能会很大，需要模运算：

```
MOD = 10**9 + 7

def build_prefix_with_mod(arr):
    n = len(arr)
    prefix = [0] * (n + 1)
    for i in range(n):
        prefix[i + 1] = (prefix[i] + arr[i]) % MOD
    return prefix
```

第五部分：常见错误与陷阱

5.1 索引错误

错误示例：

```
# ✗ 错误：忘记区分 0-indexed 和 1-indexed
prefix[i] = prefix[i-1] + arr[i] # 当 i=0 时越界
```

正确做法：预留 `prefix[0] = 0` 作为哨兵。

5.2 二维前缀和的容斥错误

常见错误：

```
# ✗ 错误：遗漏了容斥的某一项
prefix[i][j] = (matrix[i-1][j-1] +
               prefix[i-1][j] +
               prefix[i][j-1]) # 缺少减号
```

正确做法：记住完整的容斥公式。

5.3 差分更新后忘记求前缀和

```
# ✗ 错误：更新差分数组后直接使用
range_add(diff, l, r, val)
print(diff) # 这不是真实的数组值
```

```
# ☒ 正确：需要求前缀和
range_add(diff, l, r, val)
result = get_array_from_difference(diff)
print(result)
```

第六部分：性能对比

假设有 n 个元素， q 次操作（ s 次查询， u 次更新）：

方法	预处理	单次查询	单次更新	总时间
原数组	$O(1)$	$O(1)$	$O(1)$	$O(q)$
前缀和	$O(n)$	$O(1)$	$O(n)$	$O(n + s)$
差分	$O(n)$	$O(n)$	$O(1)$	$O(n + u)$
线段树	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n + q \log n)$
树状数组	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n + q \log n)$

结论：

- 当 $s \gg u$ 时，用前缀和
- 当 $u \gg s$ 时，用差分
- 当 $s \approx u$ 时，考虑线段树或树状数组

总结

1. **前缀和** 擅长范围查询，通过牺牲更新效率来换取查询效率
2. **差分** 擅长范围更新，通过牺牲查询效率来换取更新效率
3. 两者互为逆操作，都基于 **线性性** 和 **叠加原理**
4. 理解容斥原理是处理多维问题的关键
5. 实际应用中要根据操作频率选择合适的方案